

Enhanced Report: MLSS Exoplanet Habitability Classifier

Bhargav Dave

June 28, 2026

Abstract

This enhanced report documents the transformation of a notebook-only graduate MLSS exoplanet habitability classifier into a reproducible, modular, testable tabular machine learning project. The original task predicts a binary `habitable?` label from selected Kepler Object of Interest (KOI) features. The enhanced project adds dependency management, configuration, a reusable Python package, a training CLI, leakage-safe scikit-learn pipelines, synthetic-data tests, generated evaluation artifacts, and scientific caveats. Newly computed enhanced-pipeline results show that Logistic Regression with Elastic Net regularization and RBF SVM tie for the best test macro F1 score at 0.9648, while XGBoost has the best cross-validated macro F1 score at 0.9559, closely followed by Random Forest at 0.9557. These high scores should be interpreted as performance on a supplied derived label-list distinction, not proof of biological habitability.

Contents

1	Project Motivation	3
2	Dataset and Provenance	3
3	Original Notebook Workflow	3
4	Feature Space	4
5	Target Construction and Scientific Caveat	4
6	Enhanced Project Architecture	5
7	Config-Driven Workflow	5
8	Data Pipeline	6
9	Leakage-Safe Preprocessing	6
10	Model Registry and Training CLI	6
11	Testing and Validation	7
12	Historical Notebook Results	7
13	Enhanced Pipeline Results	8
14	Comparative Analysis	8

15 Confusion Matrix and Class-Specific Interpretation	8
16 ROC, Precision-Recall, and Permutation Importance	10
17 Evaluation Design	12
18 Software Engineering Improvements	12
19 Limitations	12
20 Future Work	13

1 Project Motivation

Exoplanet habitability classification is an appealing machine learning problem because it connects tabular predictive modeling with a scientifically meaningful question: which observed planet candidates resemble previously labeled habitable candidates? The project is also a useful graduate-level data science exercise because it includes messy provenance, class imbalance, feature selection, model comparison, and interpretability concerns.

The scientific framing needs care. This project does not detect life, measure atmospheric habitability, or prove surface conditions. It predicts a binary label, `habitable?`, derived from supplied habitable and non-habitable lists. The resulting models should therefore be read as classifiers of an existing label-list distinction rather than as definitive physical habitability models.

2 Dataset and Provenance

The original notebook frames the data as Kepler Object of Interest (KOI) / Kepler cumulative exoplanet data combined with separate habitable and non-habitable planet lists. The enhanced repository expects three local raw Excel files:

- `data/raw/cumulative-exoplanets.xlsx`
- `data/raw/habitable_planets_detailed_list.xlsx`
- `data/raw/non_habitable_planets_confirmed_detailed_list.xlsx`

Raw data is intentionally not committed to GitHub. During validation, the raw files were copied from the parent/original project folder into ignored local `data/raw/`. They remained ignored and were not staged.

The exact provenance and version of the three local Excel files remain uncertain unless visible from the files themselves. The NASA Exoplanet Archive provides external documentation for KOI table columns and is used here as column-definition context, not as a claim that the local Excel files were downloaded directly from a specific archive endpoint in this exact form [2].

3 Original Notebook Workflow

The original notebook is preserved at `notebooks/mlss-dacss756-final-project-bdave_v10.ipynb`. Its title currently says “Predicting the Hability of Exoplanets using Machine Learning”; the typo should eventually be corrected to “Habitability,” but the notebook itself remains unchanged.

The historical notebook workflow:

1. Load the cumulative KOI-style table and the two label-list files.
2. Start from a cumulative table visible in the notebook as 9,564 rows and 141 columns.
3. Keep `kepoi_name` and 15 numeric KOI features.
4. Construct `habitable?` by assigning 1 to the habitable list and 0 to the non-habitable list.
5. Merge the labels onto selected KOI features by `kepoi_name`.
6. Produce a modeling dataset visible in the notebook as 2,372 rows and 17 columns.

7. Remove outliers from `koi_smass` using IQR multiplier 1.5 and from `koi_prad` using IQR multiplier 2.0, leaving 2,121 rows.
8. Use an 80/20 stratified train-test split with `random_state=42`.
9. Use median imputation.
10. Compare Logistic Regression variants, Random Forest, Gradient Boosting, XGBoost, and SVM variants using macro F1.

4 Feature Space

The model uses 15 numeric features, listed in Table 1. These include planet-candidate quantities, orbital quantities, and host-star quantities.

Table 1: Selected KOI features used by the project.

Feature	Description
<code>koi_period</code>	Orbital period, usually in days.
<code>koi_ror</code>	Planet-star radius ratio.
<code>koi_srho</code>	Fitted stellar density associated with the transit model.
<code>koi_prad</code>	Planetary radius in Earth radii.
<code>koi_sma</code>	Semi-major axis in astronomical units.
<code>koi_teq</code>	Estimated equilibrium temperature in Kelvin.
<code>koi_insol</code>	Insolation flux relative to Earth.
<code>koi_dor</code>	Planet-star distance divided by stellar radius.
<code>koi_count</code>	Count of planets or KOIs associated with the system in the KOI table context.
<code>koi_steff</code>	Stellar effective temperature in Kelvin.
<code>koi_slogg</code>	Stellar surface gravity, typically in $\log_{10}(\text{cm/s}^2)$.
<code>koi_smet</code>	Stellar metallicity in dex.
<code>koi_srad</code>	Stellar radius in solar radii.
<code>koi_smass</code>	Stellar mass in solar masses.
<code>koi_eccen</code>	Orbital eccentricity.

5 Target Construction and Scientific Caveat

The target variable `habitable?` is a derived supervised label. The notebook assigns:

- 1 to rows in the supplied habitable planet list.
- 0 to rows in the supplied non-habitable confirmed planet list.

Those labels are merged onto selected KOI features by `kepoi_name`. This design is suitable for supervised learning, but it does not create a direct biological outcome. If the supplied label lists were created using variables similar to the predictors, then the model may learn the label-generation logic. This is possible target leakage in the scientific sense, even if the machine learning pipeline itself is engineered to avoid train/test preprocessing leakage.

6 Enhanced Project Architecture

The enhanced repository now separates project concerns:

- `README.md`: setup, usage, caveats, and project overview.
- `requirements.txt`: lightweight dependency management.
- `configs/default.yaml`: paths, features, target, split settings, outlier rules, scoring, and model grids.
- `src/exoplanet_classifier/`: reusable Python package.
- `scripts/train.py`: command-line training entrypoint.
- `docs/`: data provenance and scientific caveats.
- `tests/`: synthetic-data pytest tests.
- `reports/`: generated model reports and explanatory report packages.
- `figures/`: generated evaluation figures.

The project also keeps raw data, model artifacts, generated zip files, caches, and local Codex reports ignored by Git.

7 Config-Driven Workflow

The file `configs/default.yaml` defines the workflow instead of burying parameters in notebook cells. It specifies:

- Raw data paths under `data/raw/`.
- ID column: `kepoi_name`.
- Target column: `habitable?`.
- The 15 feature names.
- Test size: 0.2.
- Random seed: 42.
- Cross-validation folds: 5.
- Main scoring metric: `f1_macro`.
- Outlier rules: `koi_smass: 1.5` and `koi_prad: 2.0`.
- Bounded grids for DummyClassifier, Logistic Elastic Net, SVM RBF, Random Forest, Gradient Boosting, and XGBoost.

This makes the workflow easier to inspect, reproduce, and adjust without rewriting project code.

8 Data Pipeline

The enhanced data pipeline:

1. Checks that the three expected raw files exist.
2. Loads Excel files using pandas and openpyxl.
3. Selects `kepoi_name` plus the configured feature columns.
4. Constructs the binary `habitable?` label from the two label-list files.
5. Merges labels and features by `kepoi_name`.
6. Applies the configured IQR outlier rules.
7. Splits the cleaned table into `X` and `y`.
8. Creates a stratified train-test split with `random_state=42`.

The full run confirmed the expected processed shape:

- Prepared dataset rows after configured cleaning: 2,121.
- Training rows: 1,696.
- Test rows: 425.

9 Leakage-Safe Preprocessing

The enhanced workflow uses scikit-learn `Pipeline` and `ColumnTransformer` patterns [7, 4]. Median imputation is inside the model pipeline, not fit globally before cross-validation. Scaling is inside the pipeline for Logistic Regression and SVM, while tree-based models use median imputation without scaling.

This matters because preprocessing should be learned only from the training fold inside each cross-validation split. The model-selection step uses `GridSearchCV` with `StratifiedKFold`, so preprocessing and model fitting are evaluated together within each fold [3, 6].

10 Model Registry and Training CLI

The project includes a model registry for:

- `DummyClassifier` baseline.
- Logistic Regression with Elastic Net regularization.
- RBF SVM.
- Random Forest.
- Gradient Boosting.
- XGBoost, when installed.

The training CLI is:

```
python scripts/train.py --config configs/default.yaml --model all
```

It also supports `--model` for individual models and `--smoke-test` for lightweight validation. XGBoost was installed and ran successfully in the validation step. XGBoost is a scalable tree boosting system described by Chen and Guestrin and documented by the XGBoost project [1, 8].

11 Testing and Validation

The enhanced validation step confirmed:

- Dependencies installed successfully with `python -m pip install -r requirements.txt`.
- Python version: 3.10.5.
- Core imports passed for `numpy`, `pandas`, `sklearn`, `yaml`, `matplotlib`, and `pytest`.
- XGBoost was available and ran successfully.
- Tests passed: 7 passed in 21.13 seconds.
- Raw files were copied from the parent/original project folder into ignored local `data/raw/`.
- Raw files remained ignored and were not staged.
- Smoke training succeeded.
- Full bounded training succeeded.
- No model grids were changed during the run.

12 Historical Notebook Results

Table 2 shows historical notebook results. These are included for comparison only and were not regenerated by rerunning the notebook.

Table 2: Historical notebook model comparison.

Model	Historical CV macro F1	Historical test macro F1
Logistic Regression Elastic Net	0.9419	0.9648
Random Forest	0.9718	0.9540
Gradient Boosting / “Random Forest With GBM”	0.9720	0.9540
XGBoost	0.9647	0.9413
SVM RBF	0.9421	0.9648

13 Enhanced Pipeline Results

Table 3 shows newly computed enhanced-pipeline results from `reports/model_comparison.csv`. These metrics were generated by the modular training pipeline.

Table 3: Newly computed enhanced-pipeline model comparison.

Model	CV F1	Acc.	Prec.	Recall	Test F1	ROC-AUC	PR-AUC
Logistic Elastic Net	0.9327	0.9929	0.9553	0.9748	0.9648	0.9990	0.9809
SVM RBF	0.9356	0.9929	0.9553	0.9748	0.9648	0.9992	0.9876
Gradient Boosting	0.9372	0.9906	0.9363	0.9736	0.9540	0.9547	0.9527
Random Forest	0.9557	0.9906	0.9363	0.9736	0.9540	0.9967	0.9578
XGBoost	0.9559	0.9882	0.9323	0.9508	0.9413	0.9979	0.9582
DummyClassifier	0.4868	0.9482	0.4741	0.5000	0.4867	0.5000	0.0518

14 Comparative Analysis

Enhanced test macro F1 reproduces the historical test ranking closely. Logistic Elastic Net and SVM RBF tie for the best enhanced test macro F1 at 0.9648, matching the top historical test score. Random Forest and Gradient Boosting both reach 0.9540, and XGBoost reaches 0.9413, again matching the historical test pattern.

The enhanced CV values are generally lower than the historical notebook CV values. This is plausible because the enhanced workflow uses leakage-safer preprocessing inside scikit-learn pipelines, shuffled `StratifiedKFold`, and smaller bounded grids. The enhanced pipeline is designed for reproducibility and auditability rather than exhaustive hyperparameter search.

The DummyClassifier baseline is much worse: test macro F1 = 0.4867, ROC-AUC = 0.5, and PR-AUC = 0.0518. This confirms that the trained models are doing substantially more than majority-class prediction.

XGBoost has the best enhanced CV macro F1 at 0.9559, with Random Forest essentially tied at 0.9557. However, those models do not have the best test macro F1. This difference reinforces that model ranking is not perfectly stable on a small positive test set.

15 Confusion Matrix and Class-Specific Interpretation

The test set contains 425 rows, with 22 positive class cases visible in the classification reports. Because the positive class is small, a few predictions can materially change rare-class precision, recall, and F1.

Logistic Elastic Net and SVM RBF both report class 1 precision 0.91, recall 0.95, and F1 0.93 on 22 positive test cases. Random Forest and Gradient Boosting report class 1 precision 0.88, recall 0.95, and F1 0.91. XGBoost reports class 1 precision 0.87, recall 0.91, and F1 0.89. DummyClassifier predicts only the majority class, so class 1 precision, recall, and F1 are all 0.00.

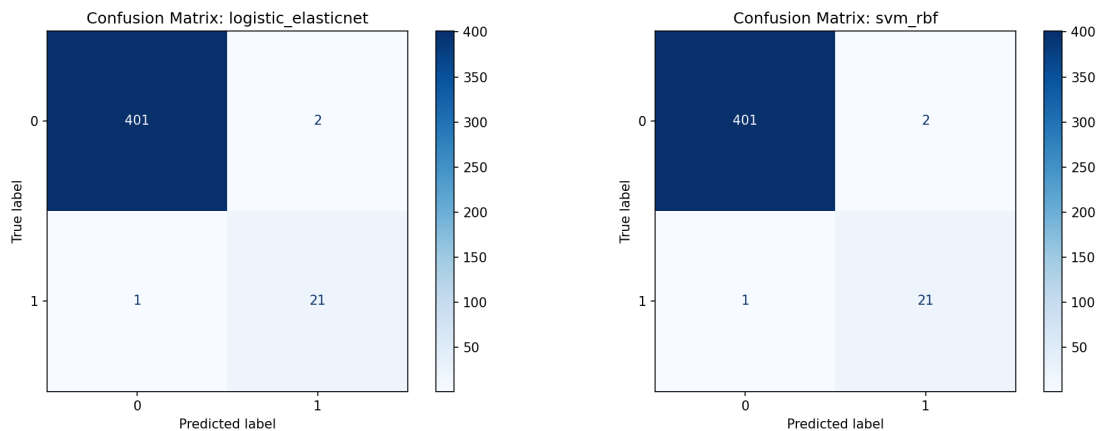


Figure 1: Confusion matrices for the two models tied for best enhanced test macro F1: Logistic Elastic Net and SVM RBF.

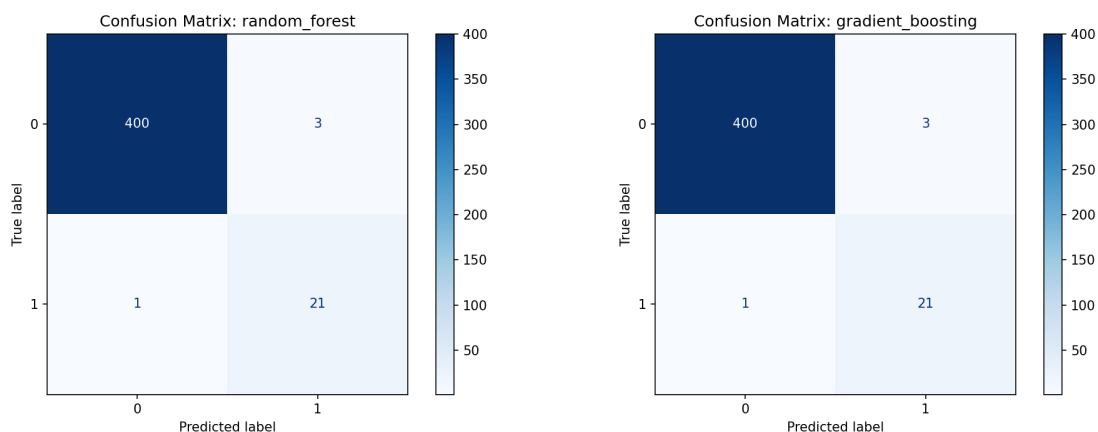


Figure 2: Confusion matrices for Random Forest and Gradient Boosting.

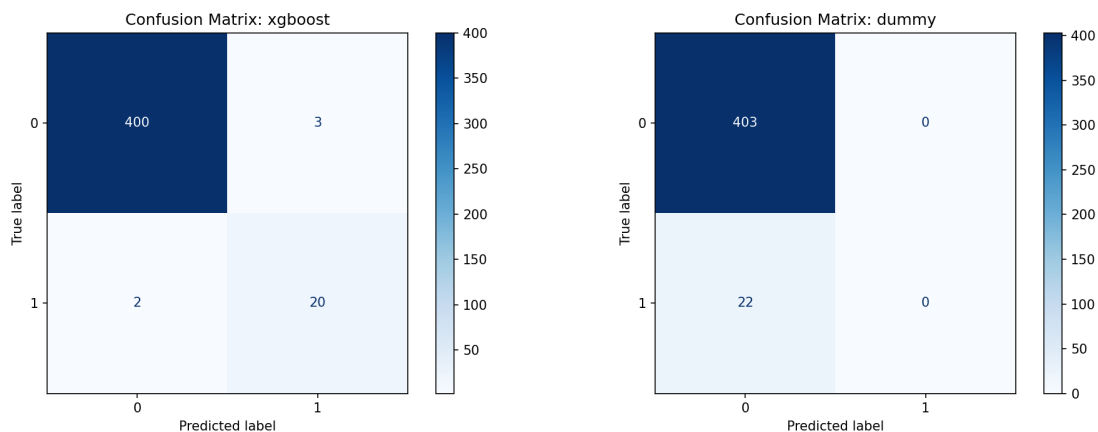


Figure 3: Confusion matrices for XGBoost and DummyClassifier. The baseline fails to identify the rare positive class.

16 ROC, Precision-Recall, and Permutation Importance

ROC-AUC is high for the trained models, but ROC curves can look optimistic under strong class imbalance. PR-AUC is especially useful here because it focuses more directly on positive-class retrieval under rare-positive conditions.

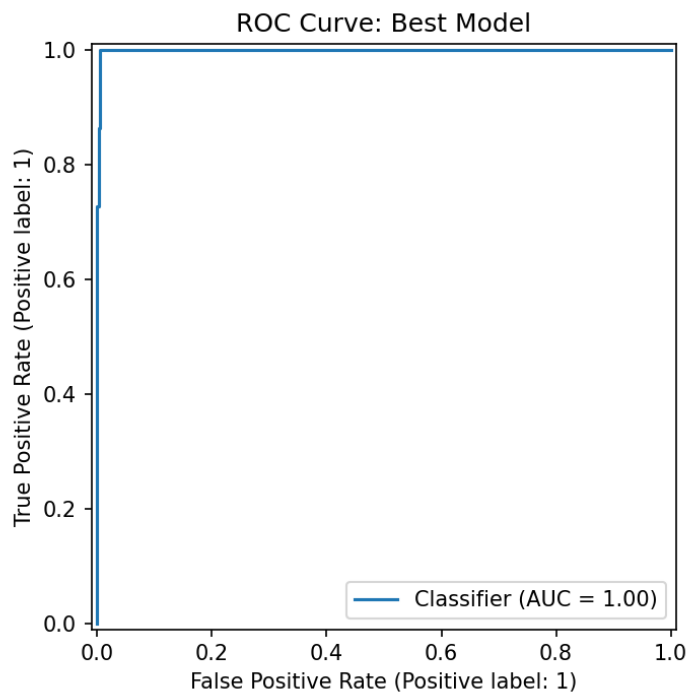


Figure 4: ROC curve for the best model selected by the enhanced training script.

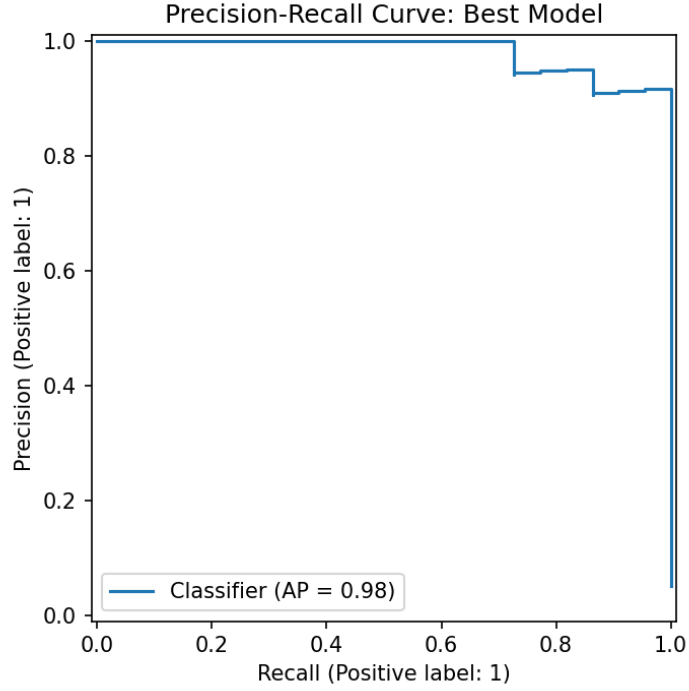


Figure 5: Precision-recall curve for the best model selected by the enhanced training script. PR-AUC is important because the positive class is rare.

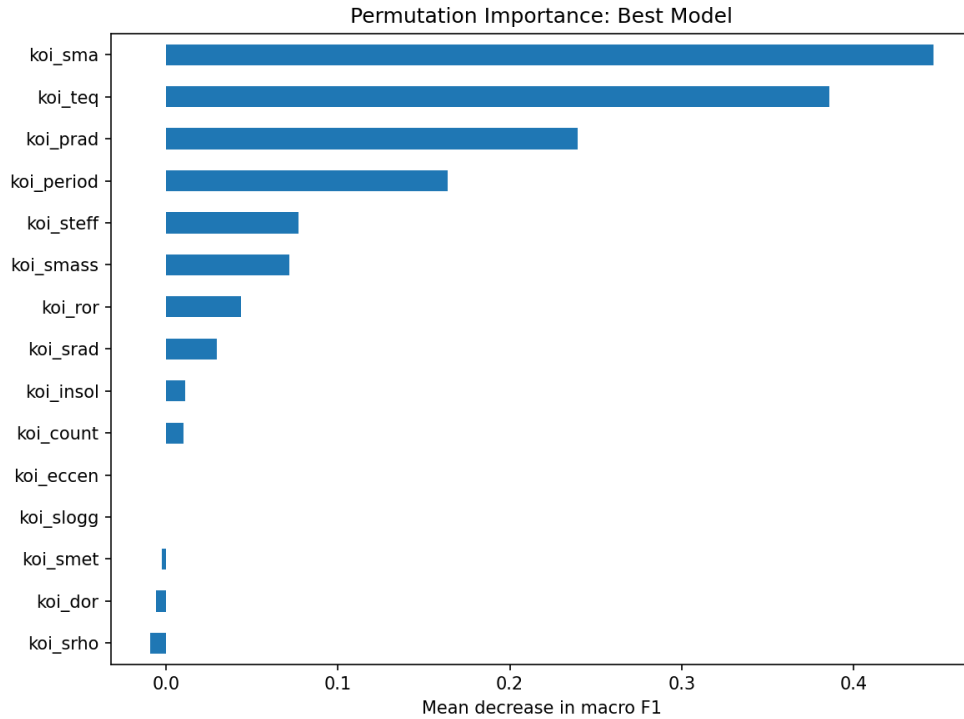


Figure 6: Permutation importance for the best model. Importance should be interpreted cautiously because features may be correlated and the target label itself is derived.

17 Evaluation Design

The enhanced pipeline reports several metrics [5]:

- Accuracy: overall fraction of correct predictions.
- Macro precision: average precision across classes.
- Macro recall: average recall across classes.
- Macro F1: average class-level F1 across classes.
- Confusion matrix: raw counts of correct and incorrect predictions by class.
- ROC-AUC: ranking quality across thresholds.
- PR-AUC: precision-recall tradeoff, especially informative for rare positive classes.

Macro F1 remains the main metric because the class distribution is highly imbalanced. Accuracy alone is misleading: the dummy baseline reaches 0.9482 accuracy by predicting the majority class, yet its macro F1 is only 0.4867 and its class 1 F1 is 0.00.

18 Software Engineering Improvements

The enhancement is primarily an engineering and reproducibility upgrade:

- Modular code replaces notebook-only logic.
- Configuration separates paths, features, split settings, model grids, and outputs.
- Tests validate imports, config loading, feature lists, preprocessing, model registry, and tiny model training.
- The training script can run smoke tests or full bounded searches.
- Raw data and generated artifacts are handled with explicit Git hygiene.
- Scientific caveats and data provenance are documented.
- Overleaf-ready reporting is separated from code execution.

19 Limitations

Important limitations remain:

- The target is a derived label, not a direct biological habitability measurement.
- The positive class is small and imbalanced.
- Test performance depends on only 22 positive examples.
- Possible label leakage exists if the label lists were created from variables similar to the predictors.
- Exoplanet measurements have uncertainty, but the pipeline uses point estimates.

- Exoplanet catalogs contain discovery and selection biases.
- The exact provenance/version of the local raw Excel files remains uncertain.
- There is no CI pipeline yet.
- There is no model card yet.
- Threshold tuning and calibration were not performed.

Most importantly, strong predictive performance may mean that the model successfully reproduces the supplied label-list distinction. It should not be presented as proof of actual planetary habitability.

20 Future Work

Recommended next steps:

- Add a verified data acquisition script if licensing and source access permit.
- Define the scientific label more rigorously.
- Add uncertainty-aware modeling or sensitivity analysis.
- Add a model card and data card.
- Add continuous integration for tests.
- Add optional experiment tracking only if it stays lightweight.
- Add threshold tuning and probability calibration.
- Use repeated cross-validation or bootstrap analysis to assess ranking stability.
- Add a small dashboard only if it supports interpretation rather than decoration.

References

- [1] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794. Association for Computing Machinery, 2016.
- [2] NASA Exoplanet Archive. *Data Columns in the Kepler Objects of Interest Table*. NASA Exoplanet Archive, IPAC, Caltech. Official KOI table column documentation. Accessed 2026-06-28.
- [3] scikit-learn developers. *Cross-validation: evaluating estimator performance*. scikit-learn. Official documentation. Accessed 2026-06-28.
- [4] scikit-learn developers. *sklearn.compose.ColumnTransformer*. scikit-learn. Official documentation. Accessed 2026-06-28.
- [5] scikit-learn developers. *sklearn.metrics.f1_score*. scikit-learn. Official documentation. Accessed 2026-06-28.
- [6] scikit-learn developers. *sklearn.model_selection.GridSearchCV*. scikit-learn. Official documentation. Accessed 2026-06-28.
- [7] scikit-learn developers. *sklearn.pipeline.Pipeline*. scikit-learn. Official documentation. Accessed 2026-06-28.
- [8] XGBoost developers. *Introduction to Boosted Trees*. XGBoost Documentation. Official documentation. Accessed 2026-06-28.